SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

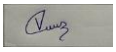
CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I Karnesh V K / 192524296 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3. An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues.
- Evaluate the advantages and limitations of serverless computing for this workload.

4. A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.
- Explain how data quality, metadata management, and governance should be maintained.
- Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5. A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference.
- Discuss the use of cloud-native AI/ML services for model deployment and monitoring.
- Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.

- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

1. A media company wants to migrate its video streaming platform to the cloud. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

Answer:

A video streaming platform must support millions of concurrent users while providing high-quality video playback with minimum buffering. A scalable cloud architecture can achieve this by combining cloud storage, Content Delivery Networks (CDNs), load balancing, auto-scaling, and monitoring services.

1. Cloud architecture design:

The proposed architecture consists of the following components:

- Video storage: Store original video files in scalable cloud object storage.
- Video processing: Convert uploaded videos into different resolutions and formats using cloud-based transcoding services.
- Content Delivery Network (CDN): Cache video content at edge locations close to users to reduce latency and buffering.
- Load balancer: Distribute incoming user requests across multiple application servers.
- Auto-scaling: Automatically increase or decrease the number of application instances according to traffic demand.
- Database: Store user profiles, subscription details, video metadata, and viewing history in a managed database.
- Monitoring: Track response time, bandwidth usage, buffering events, and server utilization.

2. Scalability and caching techniques:

Horizontal scaling can add more application instances when the number of users increases. Vertical scaling can increase the resources of existing instances when required.

CDN caching reduces repeated requests to the origin server by storing frequently accessed video content closer to users. Cache-control policies determine how long content remains cached. Frequently viewed videos can be cached for longer periods, while frequently changing information can use shorter cache durations.

Adaptive Bitrate Streaming (ABS) allows video quality to change according to the user's available bandwidth and device capabilities. This helps reduce buffering during network fluctuations.

3. Role of load balancing and auto-scaling:

Load balancing distributes incoming requests across healthy application servers. Health checks identify unavailable instances and prevent traffic from being sent to failed servers.

Auto-scaling monitors metrics such as CPU utilization, request rate, and concurrent connections. During

popular events, additional instances are launched automatically. When demand decreases, unnecessary instances are removed to reduce costs.

4. Cost optimization strategies:

- Use CDNs to reduce repeated requests to origin storage.
- Use object storage rather than expensive compute resources for video files.
- Apply compression and efficient video encoding.
- Use reserved or savings-based pricing for predictable workloads and suitable spot instances for interruptible background processing.
- Scale resources down during low-demand periods.
- Monitor bandwidth, storage, and compute costs regularly.
- Delete unnecessary temporary files and apply lifecycle policies to older content.

Conclusion:

A cloud-based video streaming architecture using CDNs, scalable storage, load balancing, auto-scaling, and adaptive bitrate streaming can support millions of concurrent users with reduced buffering. Continuous monitoring and cost optimization help maintain performance while controlling operational expenses.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

Answer:

A fintech startup using containers across multiple cloud providers must maintain consistent application behavior, secure communication, and reliable deployments. Container orchestration and a well-designed multi-cloud strategy can help manage these requirements.

1. Container orchestration solution:

Use Kubernetes or a managed Kubernetes service to deploy and manage containerized applications. Package each application with its dependencies into a container image so that it behaves consistently across environments.

Store approved images in a centralized container registry and use version tags or immutable image digests to ensure that the same application version is deployed across cloud providers.

Use Infrastructure as Code (IaC) and automated CI/CD pipelines to create infrastructure, test applications, and deploy validated releases consistently.

2. Multi-cloud deployment strategy:

Deploy Kubernetes clusters in the selected cloud providers. Use standardized configuration files,

common security policies, and automated deployment pipelines to reduce differences between environments.

Sensitive financial information should be protected through encryption, strong access controls, and appropriate regulatory compliance measures. Applications should use secure service-to-service authentication and network policies.

3. Service discovery:

Service discovery allows containers to locate and communicate with other services without requiring manually configured IP addresses.

Kubernetes Services provide stable endpoints for applications running inside a cluster. DNS-based discovery allows services to locate one another by name. For communication across different cloud providers, a service mesh or secure API gateway can provide controlled routing, authentication, and traffic management.

4. Scaling mechanisms:

The Kubernetes Horizontal Pod Autoscaler can increase or decrease the number of application pods based on CPU, memory, or custom metrics. Cluster autoscaling can add or remove worker nodes according to resource requirements.

Load balancing distributes incoming traffic across healthy application instances. During high transaction volumes, additional replicas can be deployed to maintain application responsiveness.

5. Benefits of multi-cloud deployment:

- Reduces dependence on a single cloud provider.
- Improves resilience when a provider experiences an outage.
- Allows organizations to select suitable services from different providers.
- Supports geographic distribution and disaster recovery.
- Provides flexibility in capacity and deployment options.

6. Risks and challenges:

- Different cloud providers have different APIs and management tools.
- Monitoring and troubleshooting become more complex.
- Data transfer between providers may increase costs and latency.
- Identity and access policies may become inconsistent.
- Data sovereignty and regulatory requirements must be managed.
- Running duplicate infrastructure may increase operational expenses.

Conclusion:

Kubernetes, container registries, Infrastructure as Code, and automated CI/CD pipelines provide a consistent approach to multi-cloud deployment. Service discovery, load balancing, and autoscaling improve communication and performance. However, the startup must carefully manage security, identity

federation, data consistency, monitoring, and cross-cloud costs.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. Design a robust backup and disaster recovery strategy using cloud-native backup, replication, and versioning. Explain how RTO and RPO are achieved. Provide steps to ensure business continuity during failures.

Answer:

An e-learning platform stores important information such as student records, course materials, examination results, assignments, and user accounts. Accidental deletion or infrastructure failure can interrupt learning activities and cause permanent data loss if proper protection mechanisms are not available.

A cloud-native backup and disaster recovery strategy combines automated backups, data versioning, replication, and regular recovery testing.

1. Cloud-native backup strategy:

Configure automated backups for databases, virtual machines, application configurations, and important files. Schedule backups according to the importance and frequency of data changes.

Use incremental backups to store only changes made since a previous backup, reducing storage consumption and backup time. Retain multiple backup generations to allow recovery from accidental deletion or corruption.

Store critical backups in a separate account or administrative boundary to reduce the risk of attackers or administrators deleting both production data and backups.

2. Data versioning:

Enable object versioning for course files, documents, and other supported objects. When a file is accidentally overwritten or deleted, an earlier version can be restored.

For databases, use point-in-time recovery where supported. This allows the database to be restored to a selected time before the incident occurred.

Apply retention policies and, where appropriate, immutable backup protection to prevent unauthorized modification or deletion of backup copies.

3. Data replication:

Replicate critical data to another availability zone or geographical region. Replication improves availability when the primary infrastructure becomes unavailable.

Database replication can maintain a secondary copy of important student records. Object storage replication can protect course materials and uploaded assignments.

Replication alone is not a complete backup strategy because accidental deletion or corruption may also be replicated. Independent, recoverable backup copies are therefore necessary.

4. Recovery Time Objective (RTO) and Recovery Point Objective (RPO):

- RTO: The maximum acceptable time required to restore a service after a disruption.
- RPO: The maximum acceptable amount of data loss measured in time.

For example, if the platform has an RTO of one hour, the recovery plan should restore the affected service within one hour. If the RPO is 15 minutes, the organization must design its backup and replication processes so that data loss does not exceed approximately 15 minutes under the defined recovery scenario.

Frequent backups, continuous or frequent replication, automated infrastructure deployment, and tested recovery procedures help achieve these objectives.

5. Business continuity steps:

1. Identify critical applications and classify data according to importance.
2. Define RTO and RPO for each service.
3. Configure automated backups, versioning, and replication.
4. Maintain a documented disaster recovery plan.
5. Test restoring databases, files, and applications regularly.
6. Monitor backup completion and replication health.
7. During a failure, identify the affected systems and activate the recovery plan.
8. Restore data and services, validate data integrity, and verify application functionality.
9. Inform students, teachers, and administrators about service availability.
10. Review the incident and improve the recovery process.

Conclusion:

A reliable cloud disaster recovery strategy combines automated backups, versioning, cross-region replication, and regular recovery testing. Clearly defined RTO and RPO targets help the organization select suitable recovery mechanisms. These measures protect student information, reduce downtime, and maintain continuity of online learning.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

Answer:

A hybrid cloud combines an organization's on-premises infrastructure with public cloud services. A government agency can use this model to retain sensitive applications and regulated information within its own datacenter while using cloud services for non-sensitive workloads, scalable computing, and analytics.

1. Hybrid cloud architecture:

The proposed architecture consists of the following components:

- On-premises datacenter: Hosts sensitive government applications and databases that must remain within the organization's controlled infrastructure.
- Public cloud: Hosts approved non-sensitive applications, web services, and scalable computing resources.
- Secure connectivity: A site-to-site VPN or dedicated private connection links the on-premises datacenter and cloud environment.
- Firewall and network segmentation: Separate sensitive systems from public-facing services and allow only approved communication.
- Identity provider: Provides centralized authentication and controlled access across both environments.
- Monitoring system: Collects security logs, network events, and application performance data.

2. Secure communication:

Use encrypted VPN tunnels or suitable private connectivity with additional encryption where required. Apply firewall rules and network access control policies to restrict communication between environments.

Sensitive databases should not be directly exposed to the public internet. Only approved applications and authorized services should be able to access them.

Use network segmentation to separate sensitive government records from public-facing applications. Monitor traffic for unusual activity and unauthorized access attempts.

3. Identity management:

Implement centralized identity management or secure identity federation between the on-premises directory and approved cloud services.

Use Role-Based Access Control (RBAC) to assign permissions according to job responsibilities. Apply least-privilege access and require multi-factor authentication for privileged accounts.

Regularly review user permissions and immediately revoke access when employees leave or change roles. Privileged administrative accounts should be monitored and protected with additional security controls.

4. Encryption and key management:

Encrypt sensitive data at rest using approved encryption mechanisms. Use TLS to protect information transmitted between applications, users, and cloud services.

Manage encryption keys through approved key management systems. Restrict key access, rotate keys according to policy, and maintain audit logs of key usage.

The agency should retain control over sensitive encryption keys when required by its security and compliance policies.

5. Challenges in hybrid cloud management:

- Integrating different systems, identity platforms, and management tools.
- Maintaining consistent security policies across environments.
- Meeting government data residency and regulatory requirements.
- Monitoring network connectivity and application performance.
- Managing data synchronization and transfer costs.
- Coordinating software updates and vulnerability management.
- Maintaining availability when connectivity between environments fails.

Conclusion:

A secure hybrid cloud architecture allows the government agency to retain sensitive applications on-premises while using the public cloud for approved workloads. Encrypted communication, centralized identity management, network segmentation, key management, and continuous monitoring help protect government data. Effective governance and consistent security policies are essential for managing the complexity of hybrid environments.

5. A SaaS provider needs high availability and fault tolerance. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy. Explain how data replication and failover should be configured. Discuss monitoring and alerting strategies.

Answer:

High availability ensures that an application remains accessible, while fault tolerance allows it to continue operating despite component failures. A SaaS provider can improve both by deploying application infrastructure across multiple geographical cloud regions.

1. Multi-region deployment strategy:

Deploy the application's frontend, backend services, load balancers, and supporting infrastructure in at least two cloud regions.

Each region should have sufficient capacity to serve users if the other region becomes unavailable.

Within each region, distribute resources across multiple availability zones to reduce the impact of local infrastructure failures.

Use global DNS routing or a global traffic manager to direct users to healthy regions. Health checks should detect regional failures and prevent traffic from being routed to unavailable endpoints.

2. Data replication:

Replicate application data to a secondary region using database replication and appropriate object storage replication mechanisms.

The replication strategy must consider consistency requirements, transaction volume, latency, and the acceptable amount of data loss.

For strongly consistent transactional systems, use a database architecture that supports the required

consistency guarantees. Asynchronous replication can reduce latency but may result in some recent transactions not reaching the secondary region before a failure.

Maintain independent backups because replication alone cannot protect against every type of accidental deletion or data corruption.

3. Failover mechanisms:

Configure automatic health checks to identify when the primary region is unavailable. The global traffic manager should redirect requests to the secondary region.

The failover procedure should include:

1. Detect the regional failure.
2. Confirm the health of the secondary region.
3. Promote the secondary database if required.
4. Redirect user traffic to the healthy region.
5. Validate application functionality and data consistency.
6. Notify administrators and monitor recovery progress.

Failback to the original region should occur only after the cause of the failure is resolved and data synchronization is verified.

4. Monitoring and alerting:

Monitor CPU usage, memory utilization, response time, error rate, request volume, database replication lag, and regional availability.

Use centralized logs and distributed tracing to identify errors across application components. Configure alerts for failed health checks, increased latency, abnormal error rates, and replication delays.

Create dashboards that display the health of both regions. Regularly test regional failover through controlled disaster recovery exercises.

Conclusion:

A multi-region deployment improves SaaS availability by distributing application services across geographical regions. Replication, health checks, global traffic management, and automated failover help maintain service continuity. Monitoring, independent backups, and regular disaster recovery tests are necessary to ensure that the system can recover reliably from regional outages.

SET 2

1. A retail company collects terabytes of customer transaction data every day. Design a cloud-based big data architecture using distributed storage and processing frameworks. Explain batch and stream processing, data lakes and warehouses, scalability, and cost efficiency.

Answer:

A retail company generates large amounts of data from online purchases, physical stores, payment systems, and customer interactions. An effective cloud-based big data architecture can process this

information to identify purchasing patterns, improve product recommendations, and optimize inventory management.

1. Cloud-based big data architecture:

The architecture consists of the following components:

- Data ingestion: Collect transaction data from point-of-sale systems, e-commerce applications, and other sources using APIs, message queues, and streaming services.
- Distributed storage: Store large volumes of structured, semi-structured, and unstructured data in cloud object storage.
- Distributed processing: Use Apache Spark or managed big data processing services to process data across multiple computing nodes.
- Data lake: Store raw and processed data for future analytics and machine learning.
- Data warehouse: Store cleaned, structured data for reporting, business intelligence, and analytical queries.
- Visualization: Use dashboards and reporting tools to present sales trends and inventory insights.

2. Batch and stream processing:

Batch processing analyzes large amounts of accumulated data at scheduled intervals. For example, the company can analyze daily sales to identify the most popular products and compare monthly purchasing trends.

Stream processing analyzes incoming events continuously or with low latency. It can detect sudden increases in demand, update product recommendations, and identify unusual transaction patterns.

The two methods can be integrated by sending incoming transactions to a streaming platform while storing all events in cloud storage. Stream processing provides near-real-time results, whereas batch processing performs detailed analysis over historical data.

3. Data lakes and data warehouses:

A data lake stores raw data in its original format and supports diverse analytics workloads. It can contain transaction records, customer activity logs, product information, and clickstream data.

A data warehouse stores cleaned and structured data optimized for analytical queries. It supports business reports, sales comparisons, and inventory dashboards.

Together, these systems allow the company to preserve detailed source data in the lake and provide reliable, high-performance reporting through the warehouse.

4. Scalability and cost efficiency:

- Use distributed processing to handle increasing data volumes.
- Scale computing resources according to workload demand.
- Use partitioning to improve processing and query performance.
- Compress data to reduce storage consumption.

- Apply lifecycle policies to move older data to lower-cost storage.
- Use serverless or managed analytics services where appropriate.
- Monitor resource utilization and avoid unnecessary compute capacity.
- Use incremental processing to avoid repeatedly analyzing unchanged data.

Conclusion:

A cloud-based big data architecture combining distributed storage, stream processing, batch analytics, data lakes, and data warehouses enables the retailer to analyze large datasets efficiently. These technologies improve the speed of business decisions, product recommendations, and inventory planning while supporting scalability and cost control.

2. A healthcare organization is migrating patient records and medical imaging data to the cloud. Design a secure cloud architecture. Explain encryption, IAM, key management, auditing, compliance monitoring, and methods to prevent unauthorized access and data breaches.

Answer:

Healthcare organizations store sensitive information such as patient records, prescriptions, laboratory results, and medical images. Migrating these records to the cloud requires a secure architecture that protects confidentiality, integrity, and availability while allowing authorized doctors to access information from multiple locations.

1. Secure cloud architecture:

The proposed architecture includes:

- Secure application layer: Provides doctors and authorized healthcare staff with access to patient records.
- Identity and access management: Authenticates users and determines which records they can access.
- Encrypted storage: Stores medical records and imaging data using encryption at rest.
- Secure network: Uses private networking, firewalls, and encrypted communication.
- Backup and recovery: Maintains protected backups and appropriate recovery mechanisms.
- Audit and monitoring: Records access activities and detects suspicious behavior.

Medical imaging data can be stored in scalable object storage, while patient records can be managed using an appropriate database service.

2. Encryption implementation:

Use encryption at rest for databases, storage volumes, and medical imaging files. Use TLS to encrypt communication between doctors' devices, applications, databases, and storage services.

Encryption reduces the risk of sensitive information being exposed if storage media or network traffic is compromised. Access to encrypted information must also be controlled because encryption alone does not prevent an authorized account from misusing data.

3. Identity and Access Management (IAM):

Implement role-based access control so that doctors, nurses, administrators, and technical staff receive only the permissions required for their responsibilities.

Require multi-factor authentication for sensitive accounts and privileged operations. Apply least-privilege policies and review access permissions regularly.

Where possible, use centralized identity management and secure single sign-on to manage access across healthcare applications.

4. Key management:

Use a secure key management service to create, store, control, and rotate encryption keys. Restrict key usage to authorized services and administrators.

Maintain audit records for key access and changes. Establish recovery procedures for key loss and protect keys separately from the data they encrypt.

5. Auditing and compliance monitoring:

Record patient-record access, login attempts, permission changes, data exports, and administrative actions.

Use centralized logging and automated monitoring to detect unusual access patterns, repeated failed logins, and unexpected bulk downloads.

Apply the healthcare privacy laws and security regulations relevant to the organization's jurisdiction, and maintain evidence of compliance through periodic assessments and audits.

6. Preventing unauthorized access and data breaches:

- Use strong authentication and multi-factor authentication.
- Apply least-privilege access and role-based permissions.
- Keep operating systems and applications patched.
- Encrypt sensitive information at rest and in transit.
- Use firewalls, private endpoints, and network segmentation.
- Conduct vulnerability assessments and penetration testing.
- Maintain secure backups and incident response procedures.
- Train healthcare staff to recognize phishing and social engineering attacks.

Conclusion:

A secure healthcare cloud architecture combines encryption, IAM, key management, network security, auditing, and compliance monitoring. These measures help protect patient confidentiality and prevent unauthorized access while allowing authorized healthcare professionals to retrieve medical information securely.

3. An online ticket booking platform experiences sudden spikes in traffic when popular events are announced. Design a serverless cloud architecture. Explain event-driven computing, managed databases, API gateways, message queues, and the advantages and limitations of serverless

computing.

Answer:

An online ticket booking platform may receive thousands of requests within seconds when tickets for popular concerts, sports events, or entertainment programs become available. Serverless computing can reduce infrastructure management and automatically respond to changing demand.

1. Serverless cloud architecture:

The architecture consists of the following components:

- Frontend: A web or mobile application that allows users to search for events and book tickets.
- Content Delivery Network: Delivers static website content efficiently.
- API gateway: Receives API requests and routes them to the appropriate backend functions.
- Serverless functions: Execute operations such as checking ticket availability, processing booking requests, and sending notifications.
- Managed database: Stores event details, customer information, and booking records.
- Message queue: Buffers background tasks and processes them asynchronously.
- Monitoring service: Tracks function errors, response times, request rates, and resource usage.

2. Event-driven computing:

Event-driven computing executes functions in response to events such as a booking request, payment confirmation, or ticket cancellation.

For example, when a customer completes a payment, an event can trigger a function to confirm the booking and send a notification.

Functions can scale according to incoming requests, reducing the need to maintain a fixed number of servers.

3. Role of managed databases:

A managed database stores event information, customer records, booking details, and payment status. It reduces the operational work required for database maintenance, backups, and infrastructure management.

For ticket inventory, the database must prevent two customers from purchasing the same ticket. Transactions, conditional updates, unique constraints, or other concurrency-control mechanisms should be used to maintain consistency.

4. API gateways and message queues:

The API gateway provides a controlled entry point for client requests. It can support authentication, request validation, throttling, and routing.

Message queues buffer requests when traffic temporarily exceeds processing capacity. Background functions can process notifications, emails, and other tasks without blocking the main booking request.

Queue-based processing should use idempotency controls and retry handling to avoid duplicate

bookings or repeated payment operations.

5. Advantages of serverless computing:

- Automatically scales according to demand.
- Reduces server administration.
- Charges are often based on actual usage.
- Supports rapid application development.
- Integrates well with managed cloud services.
- Simplifies event-driven application design.

6. Limitations of serverless computing:

- Cold starts can increase response time in some configurations.
- Execution duration and resource limits may apply.
- High traffic can still cause database contention.
- Vendor-specific services can increase lock-in.
- Debugging distributed functions can be complex.
- Sudden traffic spikes can increase costs if requests are not controlled.

Conclusion:

A serverless architecture using API gateways, event-driven functions, managed databases, and message queues can handle fluctuating ticket-booking traffic efficiently. Strong transaction controls, rate limiting, monitoring, and duplicate-request protection are necessary to maintain accurate ticket inventory and reliable bookings.

4. A multinational company gathers data from sales, finance, manufacturing, and customer support systems. Design a cloud-based ETL/ELT pipeline. Explain data quality, metadata management, governance, workflow orchestration, query optimization, and storage cost reduction.

Answer:

A multinational company often stores information in different systems, each with its own data format, naming conventions, and business rules. A cloud-based ETL/ELT pipeline integrates these sources into a centralized analytics platform to support consistent reporting and business decisions.

1. Cloud-based ETL/ELT architecture:

The pipeline consists of the following stages:

1. Data extraction: Collect data from sales applications, financial databases, manufacturing systems, and customer support platforms.
2. Data ingestion: Transfer data using APIs, database connectors, scheduled imports, or streaming services.
3. Landing storage: Store the original data in a cloud data lake.

4. Transformation: Clean, validate, standardize, and combine data using suitable processing tools.
5. Data loading: Load the transformed data into a cloud data warehouse or transform it within the warehouse.
6. Analytics: Connect reporting and business intelligence tools to the curated datasets.

ETL means Extract, Transform, Load, whereas ELT means Extract, Load, Transform. ETL transforms data before loading it into the target, while ELT loads data first and performs transformations within the target platform.

2. Data quality management:

Implement validation rules to detect missing values, duplicates, invalid dates, incorrect currencies, and inconsistent customer identifiers.

Standardize units, formats, and business definitions across departments. For example, sales figures from different countries may need consistent currency conversion before comparison.

Use automated quality checks and generate alerts when data fails validation.

3. Metadata management and governance:

Maintain a data catalog containing dataset descriptions, owners, formats, lineage, and access requirements.

Data lineage records where information originated and how it was transformed. This helps users trace errors and verify reports.

Apply data governance policies covering data ownership, classification, access permissions, retention, and regulatory compliance.

Use role-based access control and protect sensitive financial and customer information.

4. Workflow orchestration and scheduling:

Workflow orchestration tools coordinate extraction, transformation, validation, and loading tasks.

They manage task dependencies, scheduling, retries, failure notifications, and execution history.

For example, a daily sales report should run only after the sales data has been ingested and validated successfully.

5. Query performance optimization:

- Partition large tables by date or another suitable field.
- Use clustering or indexing features where supported.
- Avoid selecting unnecessary columns.
- Optimize joins and filter data early.
- Use materialized views or pre-aggregated tables for repeated reports.
- Process incremental changes instead of rebuilding entire datasets.

6. Storage cost optimization:

Use compression and remove redundant intermediate datasets. Apply lifecycle policies to move older

raw data to lower-cost storage.

Choose appropriate warehouse compute capacity and shut down or scale down resources when they are not required. Monitor storage growth, query costs, and unused datasets.

Conclusion:

A cloud-based ETL/ELT pipeline integrates information from multiple enterprise systems into a consistent analytics platform. Data validation, metadata management, governance, orchestration, and query optimization improve data reliability and reporting performance while reducing operational and storage costs.

5. A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. Design a cloud-based machine learning architecture. Explain big data technologies, cloud-native AI/ML services, data ingestion, feature engineering, and model updates.

Answer:

A logistics company can use machine learning to predict delivery delays by analyzing historical shipment records, current weather conditions, road traffic, and live GPS data. Accurate predictions help the company optimize delivery routes, inform customers, and improve operational planning.

1. Cloud-based machine learning architecture:

The proposed architecture includes:

- Data sources: Historical shipment records, weather APIs, GPS devices, traffic information, and delivery status updates.
- Data ingestion: Streaming services collect live GPS events, while batch pipelines import historical shipment and weather data.
- Cloud data lake: Stores raw and processed datasets.
- Big data processing: Distributed processing tools clean and transform large datasets.
- Feature engineering: Creates useful model inputs, such as distance remaining, average speed, traffic intensity, and historical route delay.
- Model training: A cloud machine learning service trains and evaluates prediction models.
- Model deployment: A model endpoint or containerized service generates predictions.
- Monitoring: Tracks prediction latency, model accuracy, data quality, and model drift.

2. Big data technologies for training and inference:

Distributed storage supports large historical datasets, while processing frameworks such as Apache Spark can clean and transform data across multiple computing nodes.

Batch processing prepares training datasets from historical shipments. Stream processing evaluates incoming GPS and traffic events to update the features used for current predictions.

The trained model can estimate the probability of a delivery delay or predict the expected arrival time.

3. Cloud-native AI/ML services:

Managed machine learning services can provide training environments, model registries, deployment endpoints, and monitoring facilities.

Models should be evaluated using suitable metrics, such as mean absolute error for arrival-time predictions or precision and recall for delay classification.

After validation, the approved model can be deployed behind an API or integrated into the logistics application. Versioning and rollback procedures help restore a previous model if the new version performs poorly.

4. Large-scale data ingestion and feature engineering:

Use message brokers or managed streaming services to receive GPS updates reliably. Partition streaming data appropriately and handle duplicate, delayed, or out-of-order events.

Clean missing values and standardize timestamps, geographical coordinates, and measurement units.

Useful features may include:

- Distance remaining to the destination.
- Average vehicle speed.
- Current weather conditions.
- Historical delay rates for a route.
- Traffic congestion levels.
- Time of day and day of the week.
- Vehicle and shipment characteristics.

Ensure that the training data uses only information that would have been available when each prediction was made. This helps prevent data leakage.

5. Model updates and monitoring:

Retrain the model periodically or when monitoring identifies significant changes in traffic patterns, weather conditions, or prediction accuracy.

Compare new models with the existing model before deployment. Use staged rollout strategies and maintain rollback capability.

Monitor data drift, model performance, prediction latency, and failures. Protect customer and shipment information through encryption and access control.

Conclusion:

A cloud-based machine learning architecture combining data lakes, streaming ingestion, distributed processing, feature engineering, and managed AI/ML services can support continuous delivery-delay predictions. Regular model evaluation, monitoring, and controlled updates help maintain prediction accuracy as logistics conditions change.

SET 3

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises datacenter to the cloud. Design a cloud migration strategy. Explain Rehost, Refactor, Replatform, Repurchase, Retire, and Retain. Discuss performance validation and methods to reduce migration risks and downtime.

Answer:

Cloud migration is the process of moving applications, databases, and IT resources from an on-premises datacenter to a cloud environment. A manufacturing company must carefully plan its migration to minimize downtime, maintain application performance, and protect business-critical information.

1. Cloud migration strategy:

The migration should be performed in stages.0

- Assessment: Identify existing applications, databases, hardware dependencies, network requirements, and security risks.
- Planning: Classify applications according to business importance, migration complexity, and compliance requirements.
- Pilot migration: Move a small, low-risk application first to validate the migration process.
- Migration execution: Transfer applications and databases using the selected migration approach.
- Testing: Verify functionality, security, performance, and data integrity.
- Optimization: Adjust cloud resources, monitor costs, and improve performance after migration.

2. Six cloud migration approaches:

a. Rehost (Lift and Shift):

Move an application to the cloud with minimal changes. This approach is suitable when the company needs to migrate quickly and avoid extensive application modifications.

b. Refactor:

Modify or redesign an application to use cloud-native capabilities such as microservices, managed databases, and serverless functions. This can improve scalability but may require significant development effort.

c. Replatform:

Make limited modifications to an application to benefit from cloud services without completely redesigning it. For example, migrate a self-managed database to a compatible managed database service.

d. Repurchase:

Replace an existing application with a different product, often a Software as a Service (SaaS) solution. This may reduce infrastructure maintenance but requires data migration and user training.

e. Retire:

Identify and remove applications that are no longer needed. Retiring unused systems reduces migration effort and operational costs.

f. Retain:

Keep certain applications on-premises temporarily or permanently because of regulatory requirements, hardware dependencies, or business constraints.

3. Performance validation after migration:

- Compare response times before and after migration.
- Perform load and stress testing.
- Monitor CPU, memory, storage, and network utilization.
- Verify database query performance.
- Test application functionality and integrations.
- Validate transaction accuracy and data integrity.
- Conduct security and vulnerability assessments.
- Test backup and recovery procedures.

4. Reducing migration risks and downtime:

Use database replication and incremental data synchronization to minimize the amount of data transferred during the final cutover.

Schedule migrations during low-activity periods. Maintain backups and a rollback plan so that the company can return to the previous environment if serious issues occur.

Use pilot migrations, automated deployment tools, and staged cutovers to reduce the impact of failures. Coordinate with manufacturing teams to ensure that critical production systems remain available.

Conclusion:

A phased cloud migration strategy allows the manufacturing company to move applications according to business requirements and technical complexity. Selecting suitable migration approaches, validating performance, maintaining backups, and preparing rollback procedures helps reduce downtime, security risks, and operational disruption.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. Design a cloud-based IoT data collection and analytics architecture. Explain message brokers, stream processing, cloud storage, high-volume data handling, scalability, reliability, and security.

Answer:

A smart city uses Internet of Things (IoT) sensors to collect information about traffic conditions, pollution, electricity usage, and public safety. A cloud-based IoT architecture can process this information in near real time and help city authorities make informed decisions.

1. Cloud-based IoT architecture:

The proposed architecture contains the following components:

- IoT sensors: Collect traffic, air quality, energy consumption, and environmental measurements.

- IoT gateways: Connect sensors to the cloud and perform initial data filtering when required.
- Message brokers: Receive and distribute sensor messages to processing applications.
- Stream processing: Analyze incoming data continuously to identify unusual conditions.
- Cloud storage: Store raw sensor data and processed results.
- Analytics platform: Generate reports, identify patterns, and support predictive analysis.
- Dashboard and alerts: Present information to city administrators and issue notifications when predefined conditions occur.

2. Message brokers and stream processing:

Message brokers receive sensor messages and deliver them to the appropriate consumers. They help separate data producers from processing applications and can buffer incoming messages during temporary processing delays.

Stream processing systems analyze incoming data continuously.

For example, traffic sensors can detect congestion, while air-quality sensors can identify pollution levels above a defined threshold. The system can send alerts to traffic management teams or environmental authorities.

3. Cloud storage integration:

Use scalable object storage or a data lake to retain raw sensor data for historical analysis. A time-series database can store recent measurements for efficient time-based queries.

Processed results can be stored in analytical databases or warehouses for reporting and visualization.

Lifecycle policies can move older data to lower-cost storage while preserving records needed for historical analysis.

4. Handling high-volume and high-velocity data:

- Use distributed message brokers to handle large numbers of incoming events.
- Partition data by sensor, location, or time.
- Use stream processing frameworks to distribute workloads across multiple workers.
- Apply filtering and aggregation to reduce unnecessary data.
- Compress historical data to reduce storage requirements.
- Use buffering and retry mechanisms to handle temporary failures.
- Monitor processing lag and increase capacity when required.

5. Scalability and reliability:

Deploy processing services across multiple instances and use autoscaling to handle increased sensor activity.

Use redundant message brokers, replicated storage, health checks, and automated recovery mechanisms.

Buffer sensor data at gateways when connectivity is temporarily unavailable, where supported.

Monitor sensor connectivity, message delivery, processing delays, and application availability.

6. Security measures:

Authenticate devices using certificates or other suitable credentials. Encrypt communication between sensors, gateways, and cloud services.

Apply least-privilege access, network segmentation, secure device updates, and regular vulnerability assessments.

Restrict access to public safety and infrastructure data. Maintain audit logs and protect sensitive location information through appropriate access controls and data retention policies.

Conclusion:

A cloud-based IoT architecture combines sensors, gateways, message brokers, stream processing, scalable storage, and analytics services. These components enable smart cities to process large volumes of data, detect problems quickly, and improve traffic management, environmental monitoring, energy efficiency, and public safety.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. Design a cloud-based business intelligence architecture. Explain data visualization integration, dashboard performance optimization, and security measures.

Answer:

Business Intelligence (BI) converts business data into meaningful reports, dashboards, and insights. A global e-commerce company can use cloud-based BI to monitor sales performance, understand customer behavior, and compare business results across different regions.

1. Cloud-based BI architecture:

The architecture consists of the following components:

- Data sources: E-commerce transactions, customer databases, inventory systems, payment records, and regional sales applications.
- Data ingestion: Collect data through APIs, database connectors, scheduled jobs, or streaming services.
- Cloud data lake: Stores raw data for historical analysis.
- ETL/ELT processing: Cleans, transforms, and integrates information from different sources.
- Cloud data warehouse: Stores structured data optimized for analytical queries.
- BI tools: Connect to the warehouse to generate interactive dashboards and reports.
- Monitoring and security: Control access and monitor system performance.

2. Data visualization and warehouse integration:

BI tools connect to a cloud data warehouse through supported connectors, SQL interfaces, or APIs.

The warehouse provides cleaned and structured information, which is displayed as charts, tables, maps, and key performance indicators.

For example, executives can use dashboards to compare regional revenue, identify popular products, track customer retention, and analyze monthly sales growth.

Filters and drill-down options allow users to examine data by region, product category, date, or customer segment.

3. Dashboard performance optimization:

- Partition large datasets by date or another suitable field.
- Optimize SQL queries and joins.
- Use pre-aggregated tables for frequently requested reports.
- Use materialized views where appropriate.
- Cache repeated query results.
- Reduce unnecessary columns and calculations.
- Schedule resource-intensive reports during off-peak hours.
- Monitor query execution time and warehouse resource utilization.

For large datasets, incremental data processing can reduce the time required to refresh dashboards.

4. Security measures:

Implement role-based access control to restrict sensitive business information.

Use row-level security to ensure that regional managers can view only the records they are authorized to access. Apply column-level restrictions where necessary to protect personal or financial information.

Encrypt data at rest and in transit. Use multi-factor authentication for privileged accounts and maintain audit logs of report access and data exports.

Regularly review user permissions and enforce appropriate data retention policies.

Conclusion:

A cloud-based BI architecture combines data ingestion, ETL/ELT processing, cloud data warehouses, and interactive visualization tools. Query optimization, caching, data aggregation, and secure access controls help executives obtain timely business insights while protecting sensitive information.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance. Design a cloud monitoring and performance optimization solution. Explain metrics, logs, tracing, resource optimization, capacity planning, availability, and cost control.

Answer:

Online gaming applications require low latency, high availability, and reliable server performance. When player numbers increase suddenly, insufficient computing resources, network congestion, database bottlenecks, or inefficient application code may reduce the gaming experience.

A cloud monitoring and performance optimization solution can identify these issues and adjust resources according to demand.

1. Cloud monitoring architecture:

The architecture includes:

- Application servers: Process gameplay requests and player sessions.
- Load balancers: Distribute traffic across healthy server instances.
- Autoscaling: Adds or removes server instances according to workload demand.
- Monitoring service: Collects performance metrics and system health information.
- Centralized logging: Stores application and infrastructure logs.
- Distributed tracing: Tracks requests across multiple application services.
- Alerting system: Notifies administrators when performance problems occur.

2. Role of metrics, logs, and tracing:

Metrics: Numerical measurements such as CPU utilization, memory usage, network throughput, request latency, error rate, and active player count. These help identify abnormal resource usage.

Logs: Records of application events, errors, failed requests, and system activities. They help administrators investigate the cause of failures.

Distributed tracing: Follows a request through multiple services and identifies where delays occur, such as an overloaded database or slow API.

Together, these monitoring methods provide a detailed view of system health and performance.

3. Resource optimization:

- Use horizontal autoscaling to add server instances during peak traffic.
- Balance player requests across healthy servers.
- Optimize database queries and connection management.
- Cache frequently accessed game data.
- Reduce unnecessary network communication.
- Profile application code to identify inefficient operations.
- Use suitable instance types based on measured workload requirements.
- Distribute infrastructure across availability zones where appropriate.

For latency-sensitive gameplay, servers should be located close to player populations whenever practical.

4. Capacity planning:

Analyze historical traffic patterns, active player counts, CPU usage, memory consumption, and network throughput.

Use load testing to determine how many concurrent players can be supported before performance degrades.

Estimate the capacity required for expected peak demand and provide suitable additional capacity for unexpected traffic increases.

5. Availability and cost optimization:

Deploy redundant application instances and use health checks to remove failed instances from service.

Configure alerts for high latency, increased error rates, resource saturation, and service failures.

Maintain backups and test recovery procedures.

Use autoscaling to reduce excess capacity during low-demand periods. Monitor cloud spending, remove unused resources, and choose suitable pricing models for predictable workloads.

Conclusion:

A cloud monitoring and performance optimization solution helps gaming companies identify bottlenecks and maintain low latency during peak hours. Metrics, logs, tracing, autoscaling, load balancing, and capacity planning improve availability and player experience while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. Design a cloud-based data lifecycle management strategy. Explain data classification, retention policies, archival storage, governance, auditing, regulatory compliance, security, and cost optimization.

Answer:

A financial institution must retain large volumes of customer transaction records for regulatory compliance, auditing, fraud investigation, and business analytics. A cloud-based data lifecycle management strategy helps the institution store, protect, access, archive, and eventually dispose of data according to business and legal requirements.

1. Cloud-based data lifecycle architecture:

The architecture consists of the following stages:

1. Data ingestion: Collect transaction records from banking applications, payment systems, and financial databases.
2. Data classification: Categorize information according to sensitivity, business value, and regulatory requirements.
3. Active storage: Store frequently accessed data in appropriate high-performance storage.
4. Processing and analytics: Analyze transaction data for reporting, fraud detection, and business intelligence.
5. Archival storage: Move eligible historical records to lower-cost storage.
6. Retention and deletion: Retain records for the required period and dispose of them securely when permitted.

2. Data classification:

Classify information into categories such as public, internal, confidential, and highly sensitive.

Customer account details, transaction histories, and personal information require stronger access controls than general business reports.

Classification determines the required encryption, access permissions, storage locations, monitoring, and retention policies.

3. Retention policies and archival storage:

Define retention periods according to applicable financial regulations, legal obligations, and organizational policies.

Frequently accessed records can remain in high-performance storage. Older records that must be preserved but are rarely accessed can be moved to lower-cost archival storage.

Use automated lifecycle policies to transition eligible data between storage tiers.

Before deleting records, verify that no legal hold, investigation, audit requirement, or retention obligation prevents deletion.

4. Governance and auditing:

Establish clear data ownership and responsibilities for managing financial information.

Maintain metadata describing data sources, formats, classifications, retention periods, and access permissions.

Record access to sensitive data, administrative changes, data exports, and retention-policy modifications. Audit logs should be protected against unauthorized alteration.

Regular compliance assessments help verify that the organization follows applicable regulations and internal policies.

5. Security measures:

- Encrypt data at rest and in transit.
- Use role-based access control and least-privilege permissions.
- Require multi-factor authentication for privileged users.
- Manage encryption keys securely.
- Monitor suspicious access and unusual data exports.
- Maintain protected backups and recovery procedures.
- Apply network segmentation and regular security updates.
- Test disaster recovery plans and incident response procedures.

6. Balancing accessibility, security, and cost:

Use tiered storage to match data access frequency with storage cost. Keep current transaction data readily available for operational systems, while moving eligible historical records to archival tiers.

Use compression and efficient file formats for analytical datasets. Partition large datasets to improve query performance and reduce unnecessary data scanning.

Monitor storage growth, retrieval charges, query costs, and unused data copies. Ensure that cost optimization does not violate legal retention, data residency, security, or availability requirements.

Conclusion:

A cloud-based data lifecycle management strategy enables a financial institution to manage petabytes of transaction data efficiently. Data classification, automated retention policies, archival storage, governance, auditing, and encryption help maintain regulatory compliance and protect sensitive information. Tiered storage and continuous cost monitoring balance data accessibility, security, and long-term storage expenses.